National University of Computer & Emerging Sciences

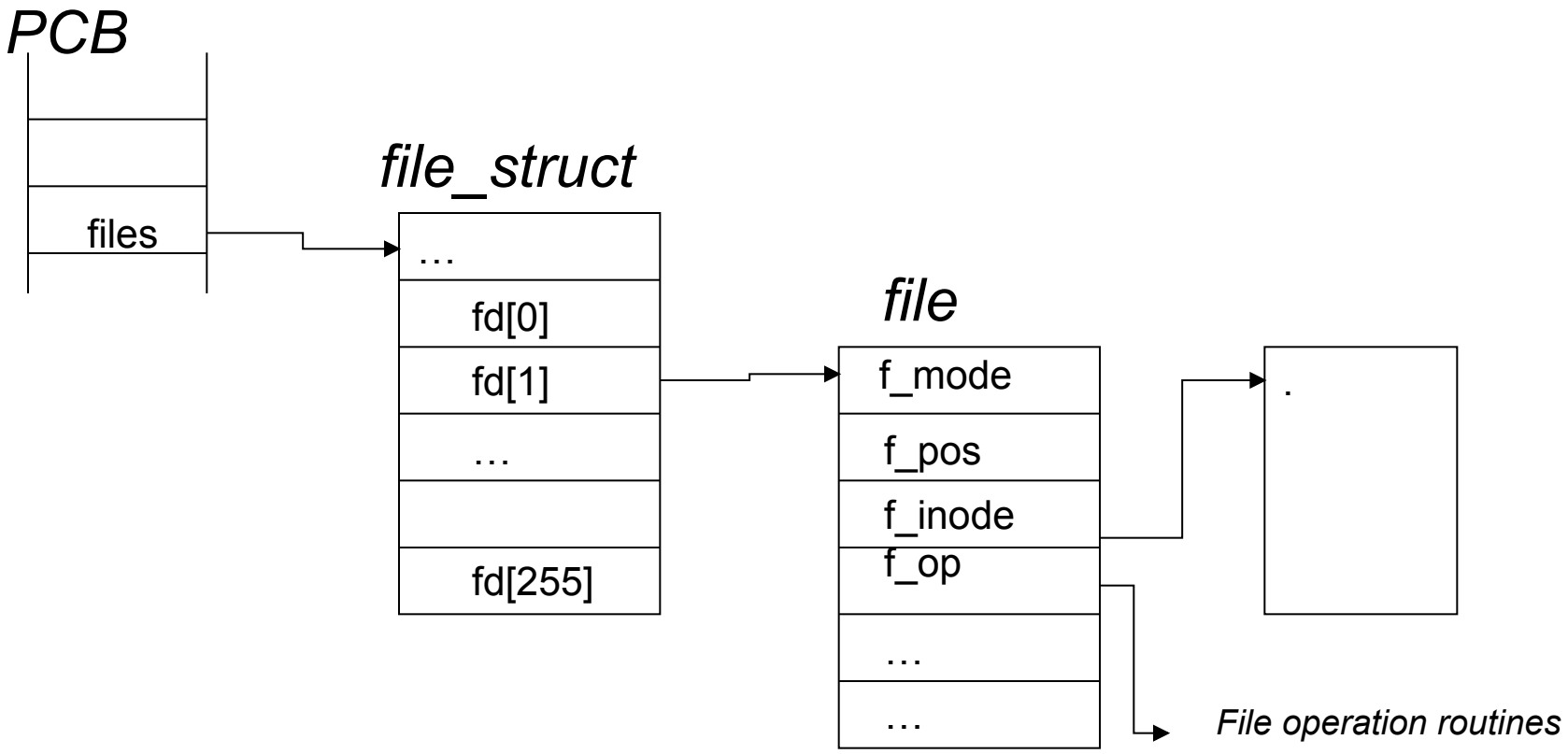
Lecture 12

IPC Unix Case Study - Pipes

Dr. Ehtesham Zahoor

File Descriptors

- The PCB of each process contains a pointer to a *file_struct, a kernel-resident array data structure containing the details of open files*



File Descriptors

- Every time a file is opened, one of the free file pointers in the *files_struct* is used to point to the new file structure.
- Linux processes expect three file descriptors to be open when they start.
- These are known as *standard input*, *standard output* and *standard error*

The Unix fact

When Unix programs do any sort of I/O, they do it by reading or writing to a file descriptor

A file descriptor is simply an integer associated with an open file

The Unix fact

A file in Unix can be

- A network connection
- A FIFO queue
- A pipe
- A terminal
- A real on-the-disk file
- Or just about anything else

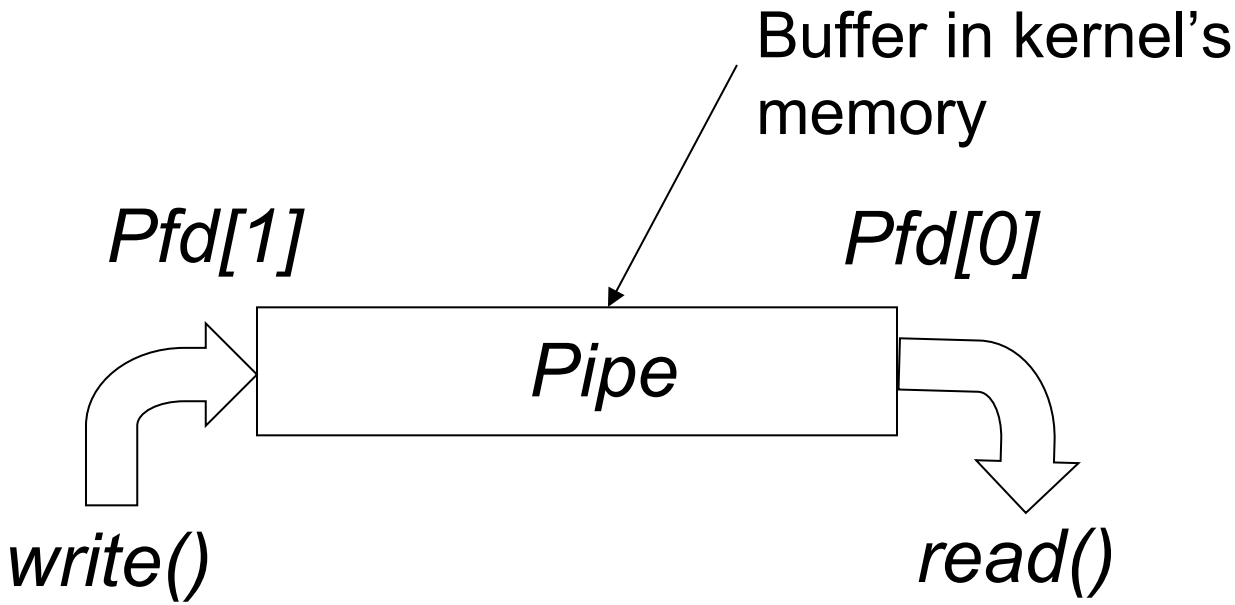
everything **in Unix is a file!**

Pipes

Provides an interprocess communication channel



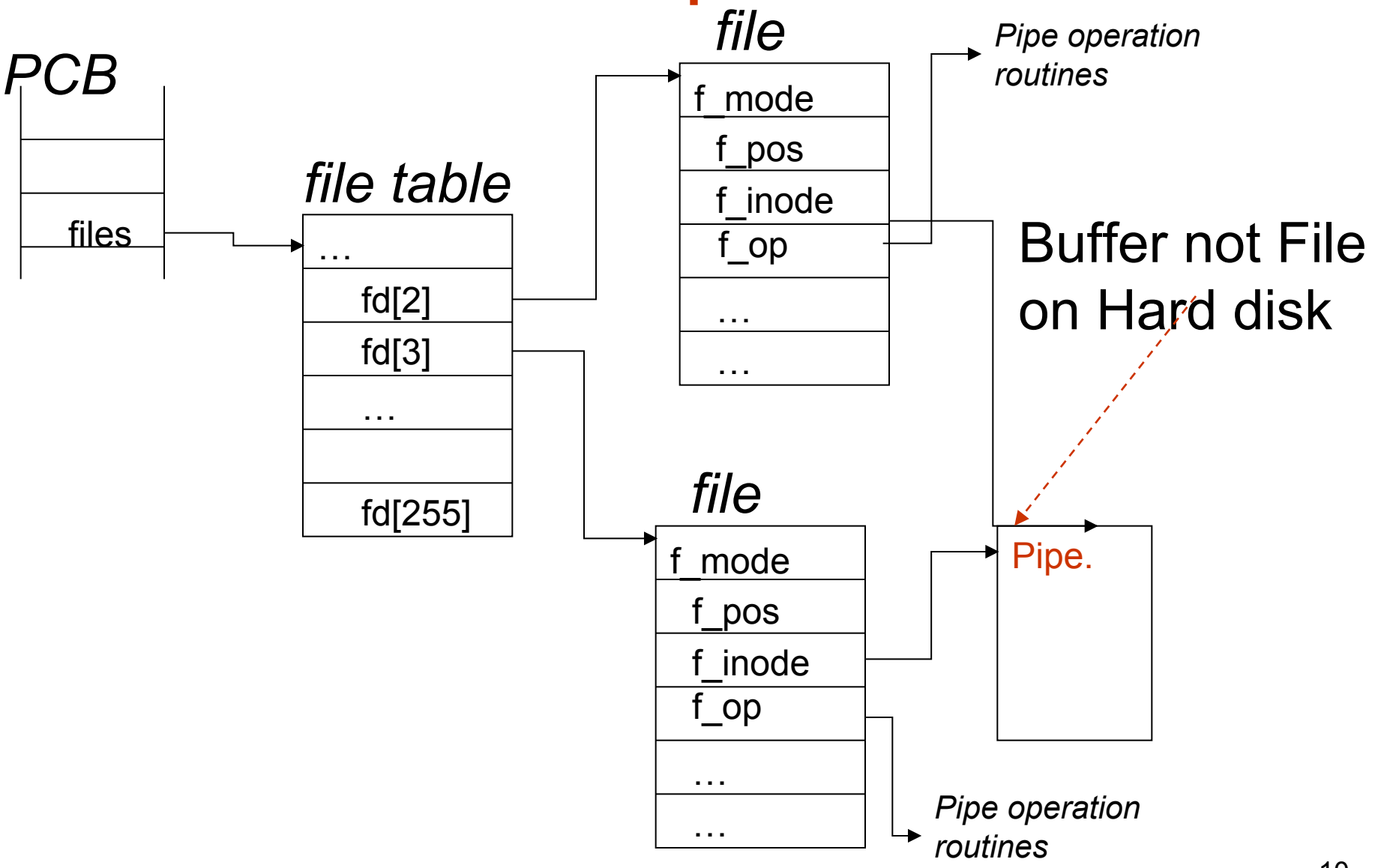
Pipes: Shared info in kernel's memory



Pipes

- A pipe is implemented using two file data structures which both point at the same temporary data node.
- This hides the underlying differences from the generic system calls which read and write to ordinary files
- Thus, reading/writing to a pipe is similar to reading/writing to a file

Pipes


PCB
files
file table
…
fd[2]
fd[3]
…
fd[255]
file
f_mode
f_pos
f_inode
f_op
…
…
Pipe operation routines
Buffer not File on Hard disk
file
f_mode
f_pos
f_inode
f_op
…
…
Pipe operation routines
Pipe.

Pipe Creation

```
#include <unistd.h>
int pipe(int filedes[2]);
```

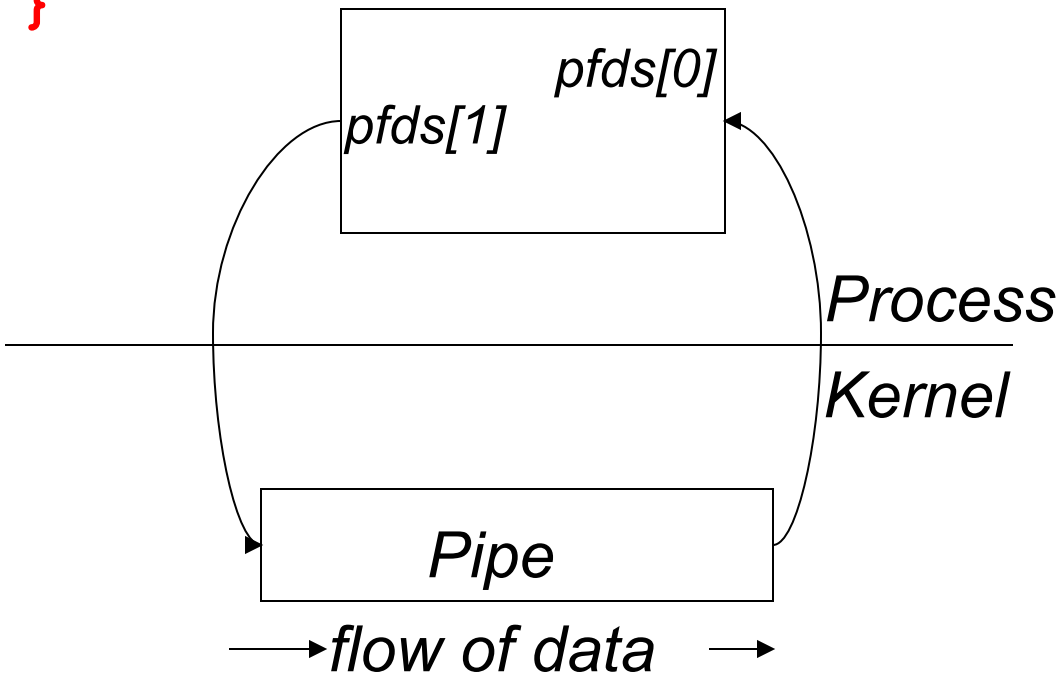
- Creates a pair of file descriptors pointing to a pipe inode
- Places them in the array pointed to by *filedes*
  - *filedes[0]* is for reading
  - *filedes[1]* is for writing.

- On success, zero is returned.
- On error, -1 is returned

Pipe Creation

```
int main()
{  int pfds[2];

if (pipe(pfds) == -1)
   {  perror("pipe");
      exit(1); }
}
```



Reading/Writing from/to a Pipe

- `int read(int filedescriptor, char *buffer, int bytetoread);`

- `int write(int filedescriptor,char *buffer,int bytetowrite);`

Example

```
int main()
{  int pfds[2];
   char buf[30];
   if (pipe(pfds) == -1) {
            perror("pipe");
            exit(1); }
   printf("writing to file descriptor #%d\n", pfds[1]);
    write(pfds[1], "test", 5);
   printf("reading from file descriptor
    write(1, "test", 5);?
   read(pfds[0], buf, 5);
   printf("read %s\n", buf);
}   read(0, buf, 5);?
```

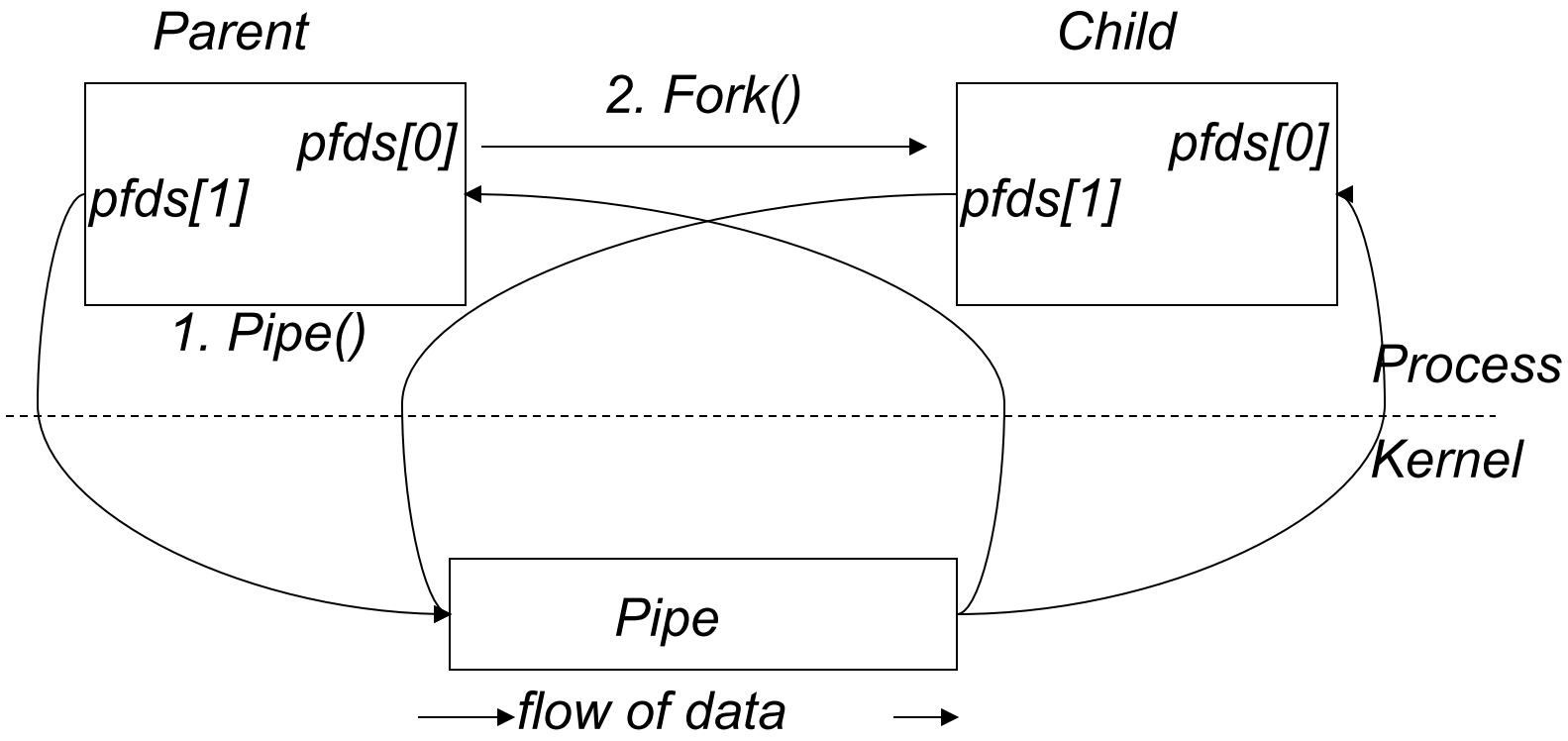
A Channel between two processes

- Remember: the two processes have a parent / child relationship
- The child was created by a fork() call that was executed by the parent.
- The child process is an image of the parent process
- Thus, all the **file descriptors** that are opened by the parent are now available in the child.

A Channel between two processes

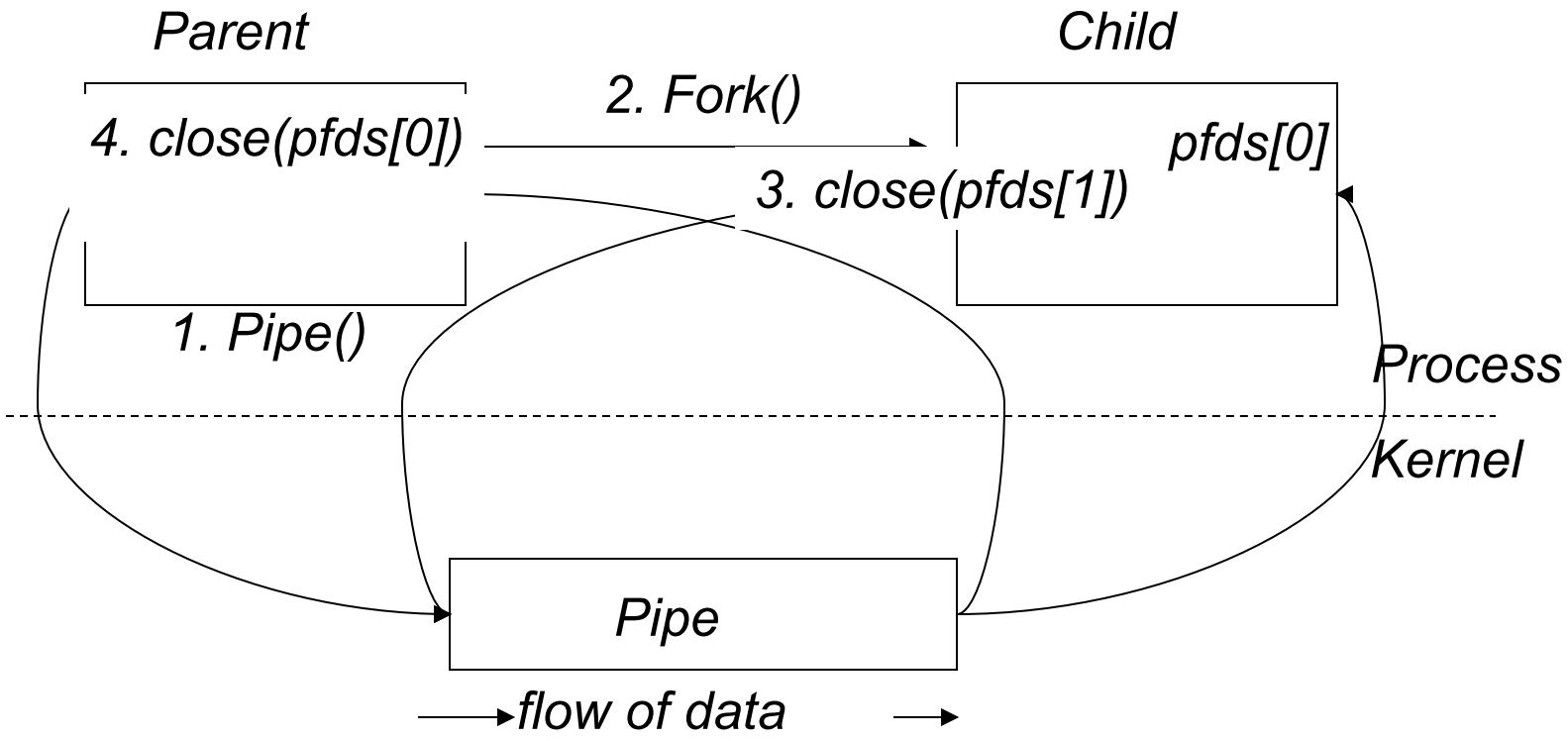
- The **file descriptors** refer to the same I/O entity, in this case a pipe.
- The pipe is inherited by the child
- And may be passed on to the grand-children by the child process or other children by the parent.

A Channel between two processes



A Channel between two processes

- To allow one way communication each process should close one end of the pipe.



Closing the pipe

- The **file descriptors** associated with a pipe can be closed with the close(fd) system call
- How would we achieve two way communication

An Example of pipes with fork

```
int main() {
   int pfds[2];
   char buf[30];
   pipe(pfds);……1
   if (!fork()) ……2
   { close(pfds[0]);……3
     printf(" CHILD: writing to the pipe\n");
     write(pfds[1], "test", 5);
     printf(" CHILD: exiting\n");
     exit(0);
   }
   else { close(pfds[1]);……4
          printf("PARENT: reading from pipe\n");
          read(pfds[0], buf, 5);
          printf( "PARENT: read \"%s\"\n", buf);
          wait(NULL);
        }
 }
```